Configuration Authentik - Instructions Détaillées

🚀 Démarrage Initial

1. Première Installation

```
# Générer les secrets
./setup_authentik.ps1  # Windows
# ou
./setup_authentik.sh   # Linux/macOS

# OU manuellement:
docker-compose up -d
```

2. Configuration Initiale Authentik

1. **Ouvrir** : http://localhost:9090/if/flow/initial-setup/
2. **Créer le compte admin** : `akadmin` / `votre_mot_de_passe_securise`

⚙️ Configuration OAuth2 Provider

Étape 1: Créer le Provider OAuth2

```
Admin Interface > Applications > Providers > Create
```

Configuration :

- **Type** : OAuth2/OpenID Provider
- **Name** : Flask IAM Dashboard
- **Authorization flow** : default-authorization-flow
- **Client type** : Confidential
- **Client ID** : `flask-iam-dashboard`
- **Client Secret** : (généré automatiquement - à noter)
- **Redirect URIs** :

  ```
  http://localhost:5000/auth/callback
  ```

Étape 2: Configurer les Scopes

Dans la section "Advanced protocol settings" :

- **Scopes** : `openid profile email groups offline_access`

- **Subject mode** : Based on the User's hashed ID
- **Include claims in id_token** : ☑ Coché

Étape 3: Créer l'Application

```
Admin Interface > Applications > Applications > Create
```

Configuration :

- **Name** : Flask IAM Dashboard
- **Slug** : `flask-iam-dashboard`
- **Provider** : Sélectionner le provider créé précédemment
- **Launch URL** : `http://localhost:5000`

👥 Configuration des Groupes et Utilisateurs

Créer les Groupes

```
Admin Interface > Directory > Groups > Create
```

Groupe Admins

- **Name** : `Admins`
- **Superuser privileges** : ☑ Coché (optionnel)
- **Attributes** :

  ```
  {
    "permissions": ["users:read", "users:update", "servers:manage"],
    "role": "Admins"
  }
  ```

Groupe read_users

- **Name** : `read_users`
- **Attributes** :

  ```
  {
    "permissions": ["users:read"],
    "role": "read_users"
  }
  ```

Créer des Utilisateurs de Test

```
Admin Interface > Directory > Users > Create
```

Utilisateur Admin

- **Username** : `admin`
- **Name** : `Administrateur`
- **Email** : `admin@example.com`
- **Groups** : Ajouter au groupe `Admins`

Utilisateur Standard

- **Username** : `user`
- **Name** : `Utilisateur`
- **Email** : `user@example.com`
- **Groups** : Ajouter au groupe `read_users`

🔧 Configuration Property Mappings

Pour que les groupes soient correctement transmis via OAuth2 :

Créer un Scope Mapping pour les Groups

```
Admin Interface > Applications > Property Mappings > Create
```

Configuration :

- **Name** : `Groups Scope Mapping`
- **Scope Name** : `groups`
- **Expression** :

```python
return {
    "groups": [group.name for group in request.user.ak_groups.all()],
    "roles": [group.name for group in request.user.ak_groups.all()],
    "permissions": sum([
        group.attributes.get("permissions", [])
        for group in request.user.ak_groups.all()
    ], [])
}
```

Assigner le Mapping au Provider

1. Aller dans le Provider OAuth2 créé
2. Section "Advanced protocol settings"

3. **Scope mappings** : Ajouter le mapping `Groups Scope Mapping`

🔗 Configuration Flask App

Mettre à jour auth.py

Remplacer dans `auth.py` ligne 15 :

```
OAUTH_CONFIG = {
    'client_id': 'flask-iam-dashboard',  # Votre vrai Client ID
    'client_secret': 'VOTRE_CLIENT_SECRET_ICI',  # Client Secret d'Authentik
    'authorization_endpoint': f"{AUTHENTIK_BASE_URL}/application/o/authorize/",
    'token_endpoint': f"{AUTHENTIK_BASE_URL}/application/o/token/",
    'userinfo_endpoint': f"{AUTHENTIK_BASE_URL}/application/o/userinfo/",
    'jwks_endpoint': f"{AUTHENTIK_BASE_URL}/application/o/flask-iam-
dashboard/jwks/",
    'scope': 'openid profile email groups offline_access',
    'redirect_uri': 'http://localhost:5000/auth/callback'
}
```

🎯 Test de la Configuration

1. Vérifier les Services

```
# Vérifier que tout fonctionne
docker-compose ps

# Logs si problème
docker-compose logs server
```

2. Test de Connexion

1. **Démarrer Flask** : `python app.py`
2. **Ouvrir** : http://localhost:5000
3. **Cliquer** : "Se connecter avec Authentik"
4. **Se connecter** avec un compte de test
5. **Vérifier** : Redirection vers le bon dashboard selon le rôle

🔍 Dépannage

Erreurs Courantes

"Invalid redirect URI"

- Vérifier l'URI exacte dans le Provider : `http://localhost:5000/auth/callback`
- Pas de slash à la fin

"Groups not found in userinfo"

- Vérifier que le scope `groups` est configuré
- Vérifier que le Property Mapping est assigné au Provider

"Access token expired"

- Utiliser le refresh token (déjà implémenté dans l'app)
- Vérifier la durée de vie des tokens dans Authentik

URLS de Debug Utiles

- **Authentik Admin** : http://localhost:9090
- **OpenID Config** : http://localhost:9090/application/o/flask-iam-dashboard/.well-known/openid-configuration
- **User Info Endpoint** : http://localhost:9090/application/o/userinfo/
- **JWKS** : http://localhost:9090/application/o/flask-iam-dashboard/jwks/

📋 Checklist de Configuration

- ☐ Authentik démarré et accessible
- ☐ Compte admin créé (`akadmin`)
- ☐ Provider OAuth2 créé avec Client ID/Secret
- ☐ Application créée et liée au Provider
- ☐ Groupes `Admins` et `read_users` créés avec attributs
- ☐ Utilisateurs de test créés et assignés aux groupes
- ☐ Property Mapping pour groups configuré
- ☐ `auth.py` mis à jour avec les vrais credentials
- ☐ Test de connexion réussi
- ☐ Permissions vérifiées selon les rôles

🎉 Configuration Terminée!

Une fois tous les éléments cochés, votre application IAM avec Authentik devrait fonctionner parfaitement. Les utilisateurs seront authentifiés via Authentik et auront accès aux fonctionnalités selon leurs groupes et permissions.

Pour des questions avancées, consulter la documentation Authentik.